**Requirements Specification**

## 1. Introduction

### 1.1 Project Goal

This project aims to develop a web application that helps users make informed decisions about movies by providing reviews and sentiment analysis. The application will retrieve user reviews from external sources, analyze them, and present a clear summary of the overall sentiment towards a movie.

### 1.2 Background

Many users struggle to choose the right movie from a vast selection. Additionally, user reviews are often scattered or difficult to interpret. By automating the analysis and summarization of reviews, this process will be simplified.

### 1.3 Scope of Application

The web application is targeted at:

- Movie enthusiasts who want an objective summary of reviews.
- Couples or groups deciding on movies to watch together.
- Cinema-goers who want to make well-informed decisions about new movies.
- Users tracking movie trends over time.

---

## 2. Requirements

### 2.1 Functional Requirements

#### 2.1.1 Retrieve Movie Information

- The application should provide a **search function** for users to find movies.
- It should display **movie details**, including:
    - Title
    - Release year
    - Average rating (e.g., IMDb, TMDb, Rotten Tomatoes)
    - Main actors
    - Director

- Genre

#### 2.1.2 User Reviews and Sentiment Analysis

- The application should **fetch user reviews** (e.g., from the TMDb API) and analyze them using text processing.
- It should calculate an **overall sentiment rating** based on positive, neutral, and negative reviews.
- Users should be able to **view historical sentiment trends** for a movie.

#### 2.1.3 Movie Comparison

- Users should be able to **compare multiple movies**.
- The application should display the sentiment and ratings for movies side by side.
- **Optional:** If two movies differ significantly, the app should suggest a **compromise movie**.

#### 2.1.4 Actor and Cast Information

- Users should be able to view a **list of actors** for each movie.
- Clicking on an actor should display a **list of movies** they have appeared in.

#### 2.1.5 Watchlist and Long-Term Tracking

- Users should be able to **add movies to a personal watchlist**.
- The application should **automatically fetch new reviews weekly** and display sentiment changes.
- **Optional:** Users should receive **notifications** when a movie's overall sentiment shifts significantly.

#### 2.1.6 User Interaction (Optional)

- Users should be able to **submit their own reviews and comments** on movies.
- The application should **suggest recommendations** based on the user's past watch history.

---

### 2.2 Non-Functional Requirements

#### 2.2.1 Performance and Scalability

- The application should be able to **handle hundreds of simultaneous requests**.
- Data should be **efficiently cached** to reduce repeated API requests.

#### 2.2.2 Security

- **API keys and user data** should be stored securely.
- If authentication is implemented, **OAuth2 or JWT** should be used for login security.

#### 2.2.3 User-Friendliness

- The web interface should be **modern and responsive** (works on mobile and desktop).
- Navigation should be **intuitive**, making it easy for non-technical users to use the app.

---

## 3. Technical Requirements

### 3.1 Technology Stack

- **Backend:** Django + Django REST Framework
- **Frontend:** React (with state management such as Redux or Context API)
- **Database:** PostgreSQL (for production), SQLite (for development)
- **APIs:** TMDb API for movie data & reviews, NLP for sentiment analysis

### 3.2 Data Model

| Entity | Attributes |
|---|---|
| **Movie** | ID, Title, Release Year, Average Rating, Genre |
| **Review** | ID, Movie ID, Author, Content, Sentiment Score |
| **Actor** | ID, Name, Movies |
| **Watchlist** | User ID, Movie ID, Date Added |

---

## 4. Exclusion Criteria

- The application will **not** create its own movie reviews.
- The application will **not** provide paid content.

- The application will **only use officially accessible APIs** and will not scrape unauthorized data sources.

---

**5. Timeline & Next Steps**

**5.1 Project Phases**

| Phase | Milestone | Expected Date |
|---|---|---|
| **Planning** | Define requirements, sketch architecture | [Date] |
| **Development (MVP)** | Implement backend API and core functionalities | [Date] |
| **Frontend Prototype** | Create React UI with search and display functions | [Date] |
| **Integration** | Connect frontend with backend | [Date] |
| **Testing Phase** | Debugging and optimization | [Date] |
| **Release** | Deploy the first version | [Date] |

**5.2 Next Steps**

- **Confirm core functionalities**
- **Create a basic UI prototype (wireframe)**
- **Define API endpoints and data structure**